

Conversions and Specification

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1	Conversions & Specification	1
1.1	Overview	1
1.2	The embedding (Fuller.4D to Coxeter.4D)	1
1.3	Gram identities and exact distances	2
1.4	Exact canonical inversion	2
1.5	Lattice context	4
1.6	API Summary	4
1.7	Verification	4

1 Conversions & Specification

1.1 Overview

Sections 2, 3 and 13 establish the Quadray/IVM framework and its computational tooling; this section documents the conversion layer that joins its two coordinate namespaces — `conversions.py`, the module that maps between Fuller.4D quadray coordinates and Coxeter.4D Cartesian coordinates. Three concerns live here: the embedding itself and its projective fiber, the Gram identities that make lattice distances exact, and an **exact rational inversion** that answers “which lattice point is this point?” with `fractions.Fraction` arithmetic instead of floating-point rounding.

The full mathematical specification of this layer is the repository-root file `SPEC.md`; the `tests/test_spec_examples.py` pins its worked examples numerically. This section is the manuscript projection of that specification: same definitions, same error taxonomy, same guarantees, prose instead of source. The analytical foundations it draws on are **Quadray Methods** (normalization, the `Quadray` class), the shell machinery of **IVM Field Learning**, and the nearest-site search of **Lattice Tooling**, whose exact-distance identity this section re-derives from the embedding’s Gram matrix.

1.2 The embedding (Fuller.4D to Coxeter.4D)

The default embedding maps the four quadray axes (A, B, C, D) to the vertices of a regular tetrahedron in $\mathbb{R}^3$. With scale factor s (uniform; scales all resulting coordinates):

$$M = s \times \begin{pmatrix} 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \end{pmatrix}, \quad \text{xyz}(q) = M q. \quad (1)$$

`urner_embedding(scale=1.0)` builds exactly this matrix (each row scaled by `scale`), and `quadray.DEFAULT_EMBEDDING` is the

same unscaled matrix stored as a tuple-of-rows; the two agree entry for entry at scale 1. Every row of M sums to zero:

$$\sum_j M_{ij} = 0 \quad \forall i, \quad \text{hence} \quad q \sim q + t \mathbf{1}, \quad t \in \mathbb{Z}, \quad (2)$$

where $\mathbf{1} = (1, 1, 1, 1)$. The vector $\mathbf{1}$ spans the kernel of M , so the map is projective: $\text{to_xyz}(q) == \text{to_xyz}(q + t * (1, 1, 1, 1))$ for every integer t , and quadray normalization (see [Quadray Methods](#)) simply selects the min-0 representative of that equivalence class. The forward map `quadray_to_xyz(q, M=None)` delegates to `quadray.to_xyz`; passing `M=None` selects `quadray.DEFAULT_EMBEDDING`, and an explicit `M` is shape-validated against the shared internal helper `_embedding_rows` (a (3, 4) array is required, `ValueError` otherwise) before the product is taken. Existing callers that pass an explicit embedding — for example `quadmath/scripts/quadray_clouds.py` — are unaffected.

1.3 Gram identities and exact distances

The embedding matrix satisfies two exact Gram identities. Row-wise, at scale s :

$$M M^T = 4 s^2 I_3, \quad (3)$$

and column-wise, with C_j the four columns of M and J_4 the all-ones matrix:

$$M^T M = 4 I_4 - J_4, \quad \text{i.e.} \quad C_i \cdot C_j = 4 \delta_{ij} - 1 \quad (s = 1). \quad (4)$$

`embedding_basis(M=None)` returns the columns of the validated embedding as a (4, 3) array B whose rows are the C_j ; at scale 1 the identity (4) reads $B B^T = 4 I_4 - J_4$ exactly (diagonal 3, off-diagonal -1), and it scales by s^2 for `urner_embedding(scale)`. This is the identity that `lattice_search.squared_distance` relies on: for integer quadrays p, q with $\delta = p - q$, the Euclidean squared distance in embedded coordinates is the exact integer

$$d^2(p, q) = \sum_i (M \delta)_i^2 = 4 \sum_j \delta_j^2 - \left(\sum_j \delta_j \right)^2, \quad (5)$$

the same identity documented as Eq. (??) in [Lattice Tooling](#), where it powers ranking and filtering; the test suite cross-validates it against float embeddings and `quadray.distance`. Because the identity is exact on integers, nearest-site decisions never suffer floating-point boundary errors.

Executable check of the two identities:

```
1 import numpy as np
2 from quadmath.lattice.conversions import urner_embedding, embedding_basis
3 from quadmath.lattice.lattice_search import squared_distance
4
5 M = urner_embedding()
6 B = embedding_basis() # (4, 3): rows are the columns of M
7 assert np.allclose(M @ M.T, 4.0 * np.eye(3)) # Eq. (eq:conv-gram-row)
8 assert np.allclose(B @ B.T, 4.0 * np.eye(4) - np.ones((4, 4)))
9 delta = np.array([2, 1, 1, 0]) # shell-1 displacement
10 assert squared_distance(delta, np.zeros((1, 4), dtype=np.int64)) == 8 # Eq. (eq:conv-distance)
```

1.4 Exact canonical inversion

The forward map is projective but not injective: by (2) the fiber of an embedded point is the full coset $\{q + t \mathbf{1} : t \in \mathbb{Z}\}$, and an inverse must select one representative from it.

$$y = (x_1, x_2, x_3, 0), \quad (C_0 \ C_1 \ C_2) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \text{xyz}, \quad q^* = \text{normalize}(y). \quad (6)$$

`xyz_to_quadray_canonical(xyz, M=None)` resolves that ambiguity deterministically by exact rational arithmetic (`fractions.Fraction`; a Python float enters as its exact binary rational — no rounding, no tolerance). The algorithm, in prose: (1) convert `xyz` and `M` to exact `Fraction` entries and require every row of `M` to sum to exactly zero, so that normalization preserves the image point (2); (2) because $C_3 = -(C_0 + C_1 + C_2)$, rank $M = 3$ is equivalent to $\det[C_0 \ C_1 \ C_2] \neq 0$ (a `ValueError` otherwise); (3) solve the 3×3 system on columns 0–2 by Cramer’s rule for the particular preimage $y = (x_1, x_2, x_3, 0)$ of (6); (4) the embedded point lies in the image of the integer lattice $\{Mq : q \in \mathbb{Z}^4\}$ if and only if x_1, x_2, x_3 are all integers — then every integer preimage is $y + t\mathbf{1}$, and the min-0 representative `Quadray(x1, x2, x3, 0).normalize()` is the unique deterministic tie-break on the fiber (2) (the same rule as `Quadray.normalize()`); a non-integral preimage raises `ValueError` (the point is not in the image).

The taxonomy is fail-closed. `ValueError` covers: `xyz` of length other than 3; `M` of the wrong shape; any row sum different from zero; $\det = 0$ (rank below 3); a non-integral preimage. `TypeError` covers entries that are not `int`/`float`/`Fraction`/`numbers.Integral` — note that `numpy.float64` (a `float` subclass) and `numpy.int64` (integral) are accepted, while `numpy.float32` is rejected. There is no floating-point fuzz anywhere: an off-lattice point raises rather than snapping to a neighbor.

The float-valued inverse `quadray.quadray_from_xyz` is a different, deliberately fuzzier contract: it applies the pseudoinverse $M^+ = M^T(MM^T)^{-1}$ and rounds each component half-up (`floor(v + 0.5)`, not banker’s rounding — its docstring argument shows why half-up provably stays in the $\mathbf{1}$ -coset of the exact preimage, where per-component round-half-even can leave it). For points on the lattice it round-trips exactly; for general $\mathbb{R}^3$ points it returns the nearest lattice point in quadray coordinates (component-wise — not always nearest in embedded XYZ distance), which is a snapping operation, not an inverse. The component-wise rule is the chosen contract; an XYZ-nearest variant would be a separate function that searches neighboring classes. `quadray_roundtrip(q, M=None)` pins the round-trip contract

$$\text{from_xyz}(\text{to_xyz}(q)) = q, \quad (7)$$

and raises `AssertionError` explicitly (not a bare `assert`, so the check survives `python -O`) when it fails. The contract is exact for normalized integer quadrays and, because `quadray_roundtrip` threads the *same* embedding through both legs, it is **scale-robust**: for $M = c M_0$ with any $c \neq 0$ the pseudoinverse projection is scale-independent,

$$\text{pinv}(cM)(cMq) = q - \frac{\sum_i q_i}{4} \mathbf{1}, \quad (8)$$

since $\text{pinv}(cM) = (1/c) \text{pinv}(M)$ and the row space of cM_0 is that of M_0 (the $(1/c) \cdot c$ cancels). A shell site embedded at scale $1/2$ — the half-integer FCC picture of the same sites — round-trips through the same scaled embedding. The single documented `AssertionError` case is an **unnormalized** input: the inverse canonicalizes to the min-0 representative, e.g. $(2, 2, 2, 1)$ comes back as $(1, 1, 1, 0)$. Embeddings whose rows do not sum to zero lie outside the Urner family of (1) and outside this guarantee.

Executable check of exact recovery and round-trips:

```

1 from fractions import Fraction
2 from quadmath.lattice.conversions import (
3     quadray_to_xyz, xyz_to_quadray_canonical, quadray_roundtrip, urner_embedding,
4 )
5 from quadmath.core.quadray import Quadray
6 from quadmath.lattice.omni_numbering import sites_through_shell
7
8 q0 = Quadray(2, 1, 1, 0) # cuboctahedron vertex, shell 1
9 assert xyz_to_quadray_canonical(quadray_to_xyz(q0)) == q0
10 assert xyz_to_quadray_canonical(

```

```

11 (Fraction(-1), Fraction(1), Fraction(1))
12 ) == Quadray(1, 1, 1, 0) # exact entries, no float fuzz
13 for site in sites_through_shell(2)[:8]: # round-trips over enumerated sites
14     assert quadray_roundtrip(Quadray(*site)) == Quadray(*site)
15 assert quadray_roundtrip(q0, erner_embedding(0.5)) == q0 # scale-robust (same rows, both legs)

```

1.5 Lattice context

The canonical representatives produced by (6) sit in the IVM site structure of **IVM Field Learning**. A normalized quadray whose component sum is $\equiv 0 \pmod{4}$ is an IVM sphere center (`is_ivm_site`); the other three residue classes are the octahedral and tetrahedral voids of the packing. The radial observable is the shell norm

$$N(q) = \sum_i |q_i - \sigma/4| = 2k, \quad \sigma = \sum_i q_i, \quad (9)$$

computed by `quadray_shell_norm`: shell k is the set of sites with $N(q) = 2k$, populated by $10k^2 + 2$ centers (Eq. (??) in **Lattice Tooling**), and `ivm_field.shell_sites(k)` enumerates shell k in lexicographic order (1, 12, 42, 92, 162 sites for $k = 0..4$). Conversions, shells, and nearest-site search share one geometry; Sections 11 and 13 detail the field learner and the query machinery respectively.

1.6 API Summary

Function	Purpose
<code>conversions.erner_embedding(scale=1.0)</code>	The (3, 4) embedding matrix of Eq. (1)
<code>conversions.quadray_to_xyz(q, M=None)</code>	Forward map via <code>quadray.to_xyz</code> ; <code>M=None</code> = <code>quadray.DEFAULT_EMBEDDING</code>
<code>conversions.xyz_to_quadray_canonical(xyz, M=None)</code>	Exact rational canonical inverse, Eqs. (2) and (6)
<code>conversions.quadray_roundtrip(q, M=None)</code>	Asserts the round-trip identity (7), scale-robust per (8)
<code>conversions.embedding_basis(M=None)</code>	(4, 3) array of embedding columns; Gram identity (4)
<code>quadray.to_xyz(q, embedding)</code>	Underlying forward product $xyz = M q$
<code>quadray.quadray_from_xyz(x, y, z, embedding)</code>	Float inverse: pseudoinverse + round-half-up (snapping, not exact inversion)
<code>lattice_search.squared_distance(p, sites)</code>	Exact squared distances via Eq. (5)

1.7 Verification

`tests/test_conversions.py` covers the layer end to end: round-trips of (7) over shells 0–4 (309 enumerated sites, center included) and at scaled embeddings per (8), exact canonical recovery of every site through (6), the Gram identities (3) and (4) with `embedding_basis`, the distance identity (5) cross-checked against float embeddings and `quadray.distance`, and the fail-closed error taxonomy (`ValueError` for off-image points, bad shapes, zero determinants and non-zero row sums; `TypeError` for inexact-foreign entry types such as `numpy.float32`). The repository-root `SPEC.md` (the full mathematical specification) and `tests/test_spec_examples.py` pin the worked examples of this section numerically as they land. Run the suite:

```

1 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run coverage run -m pytest -q
2 uv run coverage report

```